GenSpark

Control Flow: Decisions with if, elif, and else

So far, your programs run every line from top to bottom. Real programs need to choose. Is the guess too low or too high? Is the menu answer valid? Did time run out? This article teaches you to make those choices with `if`, `elif`, and `else`. It finishes the checks you started in the last article, so your program can respond to what `isdigit()` finds instead of crashing.

Comparisons Produce True or False

A decision starts with a question that has a yes or no answer. In Python, that is a comparison, and its result is a boolean: `True` or `False`.

Operator	Meaning	Example	Result
`==`	Equal to	`5 == 5`	`True`
`!=`	Not equal to	`5 != 3`	`True`
`<`	Less than	`3 < 5`	`True`
`>`	Greater than	`3 > 5`	`False`
`<=`	Less than or equal to	`5 <= 5`	`True`
`>=`	Greater than or equal to	`4 >= 5`	`False`

python

```
secret_number = 42
guess = 30
```

```
print(guess < secret_number)     # True
print(guess == secret_number)    # False
```

Notice the difference between `=` and `==`. A single `=` stores a value in a variable. A double `==` asks whether two values are equal. Mixing them up is one of the most common beginner errors.

The if Statement

An `if` statement runs a block of code only when its condition is `True`.

python

```
guess = 30
secret_number = 42

if guess < secret_number:
    print("Too low")
```

Three details matter here:

- The condition is followed by a **colon**.
- The code that belongs to the `if` is **indented**. Use 4 spaces. PyCharm does this for you when you press Enter after the colon.
- When the indented lines end, the `if` is over. Code at the original indentation runs no matter what.

python

```
if guess < secret_number:
    print("Too low")           # only runs when the condition is True
    print("Try a bigger number")  # also only runs when True
print("Thanks for playing")    # always runs
```

The indentation is how Python knows which lines belong to the `if`. Indent one line too far or not far enough, and the program breaks or behaves differently.

else: The Other Case

`else` runs when the `if` condition is `False`. It has no condition of its own.

python

```
age_text = input("How old are you? ").strip()

if age_text.isdigit():
    age = int(age_text)
    print(f"Next year you will be {age + 1}.")
else:
    print("Please enter a whole number.")
```

This fixes the crash from the last article. The program only calls `int()` after `isdigit()` has said it is safe. If the user types `"twenty"`, the `else` runs and nothing crashes.

elif: More Than Two Choices

When there are several options, add `elif` (short for "else if"). Python checks the conditions from top to bottom, runs the first block that is `True`, and skips the rest.

python

```
secret_number = 42
guess = 30

if guess < secret_number:
    print("Too low")
elif guess > secret_number:
    print("Too high")
else:
    print("Correct!")
```

This is the hint logic for the Number Guessing Game. Exactly one of the three messages prints.

You can have as many `elif` blocks as you need. The `else` at the end is optional, and it catches everything the other conditions did not.

Why elif is not the same as several ifs

Compare these two versions:

python

```
score = 85

# Version A: separate ifs, every one is checked
if score >= 60:
    print("Pass")
if score >= 80:
    print("Good")
if score >= 90:
    print("Excellent")
```

python

```
# Version B: if/elif, only the first match runs
if score >= 90:
    print("Excellent")
elif score >= 80:
    print("Good")
elif score >= 60:
    print("Pass")
else:
    print("Try again")
```

Version A prints both "Pass" and "Good" for a score of 85. Version B prints only "Good". Use `elif` when the choices are alternatives and only one should happen.

Order Matters

Because Python stops at the first match, the order of your conditions changes the result. In Version B above, if you put `score >= 60` first, then 95 would print "Pass" and never reach "Excellent". Put the most specific or most important conditions first.

This matters in the Time Challenge mode. If time has run out, the round is over, even when the guess

was correct. So the timeout check goes first:

python

```
TIME_LIMIT = 60
seconds_elapsed = 75
secret_number = 42
guess = 42

if seconds_elapsed > TIME_LIMIT:
    print("Time is up!")
elif guess == secret_number:
    print("Correct!")
elif guess < secret_number:
    print("Too low")
else:
    print("Too high")
```

The guess is correct, but this prints "Time is up!" because that condition is checked first. Move it to the bottom and the output changes. When you write a chain, ask yourself which condition should win if more than one is true.

Combining Conditions: and, or, not

Sometimes one comparison is not enough. Three keywords combine them.

Keyword	Meaning	Example	Result
`and`	Both must be true	`5 > 1 and 5 < 10`	`True`
`or`	At least one must be true	`5 < 1 or 5 < 10`	`True`
`not`	Flips True and False	`not 5 > 10`	`True`

python

```
low = 1
high = 100
guess = 150
```

```
if guess < low or guess > high:
    print("That number is out of range.")
```

The guess is out of range if it is too small **or** too big. Either one is enough.

Python also lets you write a range check the way you would say it in math:

python

```
if low <= guess <= high:
    print("In range")
```

This reads "guess is between low and high, inclusive", and it works in Python.

Use `not` with the checks you learned last time:

python

```
guess_text = input("Your guess: ").strip()

if not guess_text.isdigit():
    print("Please type a whole number.")
```

This says "if the text is not a whole number". It is easier to read than comparing with `== False`.

When you mix keywords, use parentheses. Python evaluates `not` first, then `and`, then `or`. Parentheses make your intent clear and remove the guessing:

python

```
if (guess < low or guess > high) and not is_debug:
    print("Out of range")
```

A Classic Trap with or

Beginners often write a menu check like this:

python

```
choice = "5"

if choice == "1" or "2" or "3":     # WRONG
    print("Valid")
```

It prints "Valid" for every input, including `"5"`. Python reads this as three separate conditions: `choice == "1"`, then the text `"2"`, then the text `"3"`. Any non-empty string counts as true, so the whole thing is always true.

Write the full comparison each time, or use `in`, which you learned last time:

python

```
if choice == "1" or choice == "2" or choice == "3":
    print("Valid")

if choice in ["1", "2", "3"]:        # shorter and clearer
    print("Valid")
```

Comparing Text and Numbers

A string and a number are never equal, even if they look the same:

python

```
print("7" == 7)       # False, text is not a number
print(int("7") == 7)  # True
```

This is why you convert the input before comparing it to a number. Comparing a guess typed by the user directly with `secret_number` will always say they are different.

Strings also compare character by character, not by value:

python

```
print("10" < "9")      # True, because "1" comes before "9"
print(10 < 9)          # False
```

The rule is simple: **convert to numbers before comparing numbers.**

Decisions Inside Decisions

You can put an `if` inside another `if`. This is called nesting. It is useful when the second question only makes sense after the first one is answered.

python

```
low = 1
high = 100
guess_text = input("Your guess: ").strip()

if not guess_text.isdigit():
    print("Please type a whole number.")
else:
    guess = int(guess_text)
    if guess < low or guess > high:
        print(f"Please pick a number from {low} to {high}.")
    else:
        print(f"You guessed {guess}.")
```

Read it from the outside in. First, is the text a number at all? Only if it is do we convert it, and only then do we ask whether it is in range. That order keeps `int()` from ever seeing bad text.

Each level of nesting adds 4 more spaces of indentation. Keep nesting to two or three levels. If you need more, there is usually a simpler way to write it.

Setting Variables Inside a Decision

Decisions are not only for printing. They often set values that the rest of the program uses. This is how the game turns a menu choice into settings:

python

```
choice = input("Pick a level (1 Easy, 2 Medium, 3 Hard): ").strip()

if choice == "1":
    high = 50
    max_attempts = 10
elif choice == "2":
    high = 100
    max_attempts = 8
elif choice == "3":
    high = 500
    max_attempts = 10
else:
    print("That is not a valid level.")
```

After this runs, `high` and `max_attempts` hold the right numbers for the level. One thing to notice: if the user types `"5"`, the `else` runs and neither variable is set. In the next article, you learn loops, which let the program ask again until the answer is valid.

Putting It Together

This short program uses every idea in the article. It checks a guess and gives a hint once:

python

```
secret_number = 42
low = 1
high = 100

guess_text = input(f"Guess a number from {low} to {high}: ").strip()

if not guess_text.isdigit():
    print("That is not a whole number.")
else:
    guess = int(guess_text)
    if guess < low or guess > high:
        print("That number is out of range.")
    elif guess < secret_number:
        print("Too low")
    elif guess > secret_number:
```

```
        print("Too high")
    else:
        print("Correct!")
```

Run it a few times with different input: a good guess, a number that is too high, letters, a blank entry, and `0`. It handles all of them without crashing, but it only asks once. A loop in the next article will keep it asking.

Common Mistakes

Mistake	What happens	Fix
Using `=` instead of `==` in a condition	`SyntaxError`	Use `==` to compare
Forgetting the colon after `if`, `elif`, or `else`	`SyntaxError`	End the line with `:`
Wrong or missing indentation	`IndentationError` or the wrong lines run	Indent the block 4 spaces
`if x == 1 or 2`	Always true	Write `x == 1 or x == 2`, or use `in`
Comparing text to a number	Never equal	Convert with `int()` first
Several `if`s instead of `elif`	More than one block runs	Use `elif` for alternatives
Conditions in the wrong order	The wrong block wins	Put the most important check first
Writing `else` with a condition	`SyntaxError`	`else` never has a condition. Use `elif`.

Try It Yourself

Create a file called `decisions.py` and run it with `python3 decisions.py`. Try these:

1. Ask for a number and print whether it is positive, negative, or zero. Check that the text is a number first. (Hint: `isdigit()` rejects negatives, so for this exercise type only positive numbers or zero.)

2. Ask for a score from 0 to 100 and print a grade: 90 or above is A, 80 to 89 is B, 70 to 79 is C, anything lower is "Try again".
3. Ask for a menu choice of 1, 2, or 3 and print a different message for each. Print an error message for anything else.
4. Ask for a number and print "even" or "odd". (Hint: the `%` operator gives the remainder.)
5. Ask for a number from 1 to 10. Print "Out of range" if it is outside that range, and "Valid" if not. Use both the `or` version and the `low <= x <= high` version.

Test each one with good input and with bad input, such as letters, a blank entry, and a number just outside the limit.

Why This Matters for an FDE

Every piece of client software is a pile of decisions: is this record valid, is this user allowed in, did this request time out, which handler should run? Most production bugs come from conditions that are in the wrong order, miss an edge case, or assume that input is clean. Learning to read a chain of `if` and `elif` carefully, and to ask "what if the input is something I did not expect", is a habit you will use every day.

Comprehension Check

1. What is the difference between `=` and `==`?
2. Look at the two score examples (Version A and Version B). Why does Version A print two messages for a score of 85, while Version B prints one?
3. In the timeout example, the guess is correct but the program prints "Time is up!". Why? What would you change to make a correct guess win?
4. Why is `if choice == "1" or "2" or "3":` always true? Write two correct versions.
5. What does `"7" == 7` return, and why? What does `"10" < "9"` return, and why?
6. In the nested example, why does the program check `isdigit()` before it calls `int()`?

← Previous: Input, Output, Type Conversion, and Checking Text

Next: Loops: while, for, break, and continue →

0 comments so far

6

Add a comment...

Comment cannot be empty

Comment

No comments yet. Be the first to start the discussion!